# Lab 10 – Objects and Classes Part 2

Topics Covered:

- Objects and Classes

**<u>Part 1: Pre-Lab Questions</u>**

*Question 1:* Look at the following class, called "Banana". Each time a banana is created the variable 'bananaID' is incremented. How would you create 2 banana objects (called b1 and b2) of this class?

```
class Banana:
    bananaID =0
    def __init__(self):
        Banana.bananaID += 1
        self._ID = Banana.bananaID
    def __str__(self):
        return "This banana has an id of "+ str(self._ID)
```

a) Banana(b1, b2)

b) b1 = Banana()
   b2 = Banana()

c) new Banana() = b1
   new Banana() = b2

*Question 2:* How would you print the Banana objects from the previous question?

a) print(All Bananas)

b) print(b1)
   print(b2)

c) print Banana()

*Question 3:* After creating two objects (b1 and b2) of Banana what will the output of `print(Banana.bananaID)` be?

a) 0

b) 1

c) 2

d) 3

*Question 4:* After creating objects b1 and b2 what will the output of `print(b1)` be?

a) This banana has an id of 0

b) This banana has an id of 1

c) This banana has an id of 2

d) This banana has an id of 3

*Question 5:* After creating objects b1 and b2 what will the output of `print(b2)` be?

a) This banana has an id of 0

b) This banana has an id of 1

c) This banana has an id of 2

d) This banana has an id of 3

# Lab 10 – PART 2

***1. Activity:***

For this task you will create a class car with an instance variable called **_type**. You will also **create the getter and setter for this class**. The _type should start as an empty string when an instance of the class is first created, and the getter and setter should return and set the value of _type.

```
class Car:
    def __init__(self):
        Add statement(s)

    def setType(self,type):
        Add statement(s)

    def getType(self):
        Add statement(s)

# Create three cars
c1 = Car()
c2 = Car()
c3 = Car()

# Set their types
c1.setType("Toyota")
c2.setType("Honda")
c3.setType("Nissan")

# Print the types
print(c3.getType())
print(c2.getType())
print(c1.getType())
```

***What is the output of the 3 print statements after you add the missing statements?***

2. **Follow the instructions below to construct a complete program.**

In this walkthrough we will talk about special methods with classes.

Create a class named Coffee. Coffee is a simple class with two instance variables that are set by it's constructor (the __init__ method). It also has a __str__ and __repr__ method so we can convert it to a string.

```
class Coffee:
    def __init__(self, vol, cost):
        self._volume = vol
        self._price = cost

    def __repr__(self):
        return f"Coffee({self._volume}, {self._price})"

    def __str__(self):
        return f"This coffee costs ${self._price} and is {self._volume}ml"
```

Now, create two Coffee objects:

```
small_coffee = Coffee(120, 1.50)
large_coffee = Coffee(360, 2.25)
```

Let's try printing these classes:

```
print(small_coffee)
print(large_coffee)
```

Note the output:

```
This coffee costs $1.5 and is 120ml
This coffee costs $2.25 and is 360ml
```

Passing an object of Coffee to the built-in print functions causes it to attempt to convert the object into a string (just like calling str(small_coffee)). This causes the __str__ special method to be called to create a string representation of the object.

But what happens if they are printed in a list?

```
lst = [small_coffee, large_coffee]
print(lst)
```

The output is now:

```
[Coffee(120, 1.5), Coffee(360, 2.25)]
```

When the print function is passed a sequence of objects, instead of converting them to strings using the string representation (created by the __str__ special method) it uses the representation created by the __repr__ special method. You can also get this representation by using the repr() function such as repr(small_coffee).

The difference between __str__ and __repr__ is that __str__ should create a human readable string representation of the object suitable for users of your program, while __rep__ should create a string representation of the object that is unambiguous, has the details needed to recreate the object, and is intended for developers, debugging, and logging.

Another interesting thing to try is what happens when we try other operations on objects. For example, what happens if we try to add these objects together? Give this a try:

```
print(small_coffee + large_coffee)
```

We get a type error that says:

```
TypeError: unsupported operand type(s) for +: 'Coffee' and 'Coffee'
```

Since Coffee is not a built in Python type like int and string, it doesn't support addition. This is where we can use a special method like __add__ that allows us to define what happens when two objects are added to each other. Modify the class Coffee and add the following method:

```
def __add__(self, other):
    if isinstance(other, Coffee):
        new_coffee = Coffee(self._volume + other._volume,
                            self._price + other._price)
        return new_coffee
    else:
        raise TypeError("Unsupported operand type(s) for +")
```

Now, when the + operator is used with the Coffee object it will call the __add__ function. We created an if statement to check that if the Coffee is being added with another Coffee object, the result will be a new Coffee object that contains the combined volume and price. However, if the Coffee object is added to a different object type the result will be a TypeError.

Let's now run the full code and see what output we get (record your output):

```
class Coffee:
    def __init__(self, vol, cost):
        self._volume = vol
        self._price = cost

    def __repr__(self):
        return f"Coffee({self._volume}, {self._price})"

    def __str__(self):
        return f"This coffee costs ${self._price} and is {self._volume}ml"

    def __add__(self, other):
        if isinstance(other, Coffee):
            new_coffee = Coffee(self._volume + other._volume,
                                self._price + other._price)
            return new_coffee
        else:
            raise TypeError("Unsupported operand type(s) for +")


small_coffee = Coffee(120, 1.50)
large_coffee = Coffee(360, 2.25)

print(small_coffee + large_coffee)
```

**3. Add more special methods.**

__add__ and __init__ are not the only special methods Python allows us to define. Add the special methods shown in the table below to the Coffee class from question 2.

| Special Method | Purpose | What it should do for the Coffee class. |
|---|---|---|
| **`__eq__(self, other)`** | This defines the == operation on the object. It checks if this and another object are equal. Returns True if they are and False otherwise. | Check if other is an instance of Coffee, if it is and it has the same volume. If it is not an instance of Coffee or does not have the same volume return False. |
| **`__le__(self, other)`** | This defines the <= operation on the object. It checks if this object is less than or equal to another object. | Check if the volume of this Coffee object is less than or equal to the volume of the other object. |
| **`__lt__(self, other)`** | This defines the < operation on the object. It checks if this object is less than to another object. | Check if the volume of this Coffee object is less than the volume of the other object. |

| | | |
|---|---|---|
| **__ge__(self, other)** | This defines the >= operation on the object. It checks if this object is greater than or equal to another object. | Check if the volume of this Coffee object is greater than or equal to the volume of the other object. |
| **__gt__(self, other)** | This defines the > operation on the object. It checks if this object is greater than or equal to another object. | Check if the volume of this Coffee object is greater than the volume of the other object. |

Also add some code to test each operation and print the result for example for the == operation:

```
small_coffee = Coffee(120, 1.50)
large_coffee = Coffee(360, 2.25)
same_coffee = Coffee(120, 1.00)

print("Testing small_coffee == large_coffee:", small_coffee == large_coffee)
print("Testing same_coffee == small_coffee:", same_coffee == small_coffee)
```

Which should output:

```
Testing small_coffee == large_coffee: False
Testing same_coffee == small_coffee: True
```

Make sure you also test <=, <, >=, and >.